

Assignment 3

Youssef Hossam Eldeen Mohamed Elzainy Mostafa - 212600

Part one

Q1 - Syntax Troubleshooting and Error Identification

```
module memory_ctrl (

    //-----ERROR-----
    [7:0] data_in, // unidentified dircetion port
    //-----ERROR-----

    output reg [0:15] addr_bus,
    inout data_bus, // never used in the code
    input wire clk // never used the clock in the code

);

    // Value Never changed
    // not an error but will be optimized by the synthesis tool
    wire [31:0] full_word;

    reg [7:0] grid_mem [0:3][0:1];
    reg [4:0] start_idx;

    always @(*) begin

        // Operation A
        grid_mem[0] = 8'hFF;

        //-----ERROR-----
        // Operation B
        grid_mem[1][2:3] = 8'h00;
        /*
            -- ERROR 1
            Size mismatch - 2 columns
            -- ERROR 2
        */
    end
endmodule
```

```

        can't use two columns at the same time
        (only one address is allowed at a time)
    */
    //-----ERROR-----

    // Operation C
    addr_bus[0:3] = full_word[start_idx + 3 : start_idx];
    // Error in the syntax of the variable part select
    // should be +- or -:
    // the correct syntax should be
    // full_word [start_idx +: width]
    // Value of full_word never changed wich will output ZZZZ

end

endmodule

```

Q2

- The standard binary states can't be shared by multiple devices at the same data bus at the same time → will make a short circuit which may burn your whole chip
- that's why tri state buffers are used - because it can be used to control whether the device can drive the data bus or not

```

module Tri_State_Buffer (
    inout [7:0] shared_bus,
    input [7:0] Tx_data,
    input Tx_en
);

    assign shared_bus = Tx_en? Tx_data : 8'hzz;

endmodule

```

```

module Tri_State_Buffer_tb;

    wire [7:0] shared_bus;
    reg [7:0] Tx_data;
    reg Tx_en;

    Tri_State_Buffer dut (
        .shared_bus(shared_bus),

```

```

        .Tx_data(Tx_data),
        .Tx_en(Tx_en)
    );

    initial begin
        repeat(20) begin
            Tx_data = $random;
            Tx_en = $random;

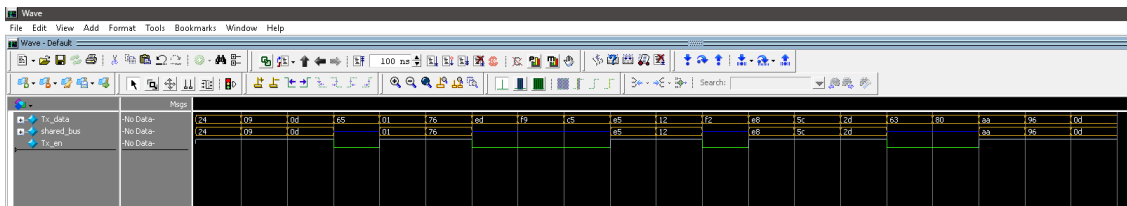
            #10;

            if (!Tx_en && shared_bus !== 8'hzz) begin
                $display("Error");
                $finish;
            end
        end

        $finish;
    end

endmodule

```



Q3

The main difference between wire and reg is where each one can be used without getting an error after compiling the code

- The reg is a variable connection and can only be used as an output in the procedural block like the always block for an example and if u used it in a continuous assignment for example it should get u an error and it's initial value is X
- while the wire is a net connection can only be used with the inputs and outputs if you're updating these outputs using a continuous assignment and it's initial value will be Z

Part 2

Task one

```
module data_router(  
    input [31:0] data_in,  
    output reg [7:0] byte_high,  
    output [7:0] byte_low,  
    output parity_bit  
);  
  
    assign byte_low = data_in[7:0];  
    assign parity_bit = ^data_in;  
    // reductional XOR is used to get the parity bit  
  
    always @(*) begin  
        byte_high = data_in[31:24];  
    end  
  
endmodule
```

```
module data_router_tb;  
    reg [31:0] data_in;  
    wire [7:0] byte_high, byte_low;  
    wire parity_bit;  
  
    data_router dut (  
        .data_in(data_in),  
        .byte_high(byte_high),  
        .byte_low(byte_low),  
        .parity_bit(parity_bit)  
    );  
  
    initial begin  
  
        repeat(30) begin  
            data_in = $random;  
  
            #10;  
  
            $monitor("data in = %h, byte high = %h, byte low = %h, parity bit  
= %b", data_in, byte_high, byte_low, parity_bit);  
        end  
  
        $finish;  
    end  
end
```

endmodule

```

VSIM 11> run -all
# data in = c0895e81, byte high = c0, byte low = 81, parity bit = 0
# data in = 8484d609, byte high = 84, byte low = 09, parity bit = 1
# data in = b1f05663, byte high = b1, byte low = 63, parity bit = 0
# data in = 06b97b0d, byte high = 06, byte low = 0d, parity bit = 0
# data in = 46df998d, byte high = 46, byte low = 8d, parity bit = 0
# data in = b2c28465, byte high = b2, byte low = 65, parity bit = 1
# data in = 89f75212, byte high = 89, byte low = 12, parity bit = 1
# data in = 00f3e301, byte high = 00, byte low = 01, parity bit = 0
# data in = 06d7cd0d, byte high = 06, byte low = 0d, parity bit = 0
# data in = 3b23f176, byte high = 3b, byte low = 76, parity bit = 0
# data in = 1e8dcd3d, byte high = 1e, byte low = 3d, parity bit = 0
# data in = 76d457ed, byte high = 76, byte low = ed, parity bit = 0
# data in = 462df78c, byte high = 46, byte low = 8c, parity bit = 1
# data in = 7cfde9f9, byte high = 7c, byte low = f9, parity bit = 1
# data in = e33724c6, byte high = e3, byte low = c6, parity bit = 0
# data in = e2f784c5, byte high = e2, byte low = c5, parity bit = 1
# data in = d513d2aa, byte high = d5, byte low = aa, parity bit = 0
# data in = 72aff7e5, byte high = 72, byte low = e5, parity bit = 0
# data in = bbd27277, byte high = bb, byte low = 77, parity bit = 0
# data in = 8932d612, byte high = 89, byte low = 12, parity bit = 1
# data in = 47ecd8bf, byte high = 47, byte low = 8f, parity bit = 0
# data in = 793069f2, byte high = 79, byte low = f2, parity bit = 0
# data in = e77696ce, byte high = e7, byte low = ce, parity bit = 0
# data in = f4007ae8, byte high = f4, byte low = e8, parity bit = 0
# data in = e2ca4ec5, byte high = e2, byte low = c5, parity bit = 0
# data in = 2e58495c, byte high = 2e, byte low = 5c, parity bit = 0
# data in = de8e28bd, byte high = de, byte low = bd, parity bit = 0
# data in = 96ab582d, byte high = 96, byte low = 2d, parity bit = 0
# data in = b2a72665, byte high = b2, byte low = 65, parity bit = 0
# ** Note: $finish      : D:/NTI - Digital IC Design/Session 4/Assignment/data router design/data_router_tb.v(23)
# Time: 300 ns  Iteration: 0  Instance: /data_router_tb
# 1

```

Signal	Value
data_in	c081
byte_high	c0
byte_low	81
parity_bit	0
data_in	8409
byte_high	84
byte_low	09
parity_bit	1
data_in	b163
byte_high	b1
byte_low	63
parity_bit	0
data_in	060d
byte_high	06
byte_low	0d
parity_bit	0
data_in	468d
byte_high	46
byte_low	8d
parity_bit	0
data_in	b265
byte_high	b2
byte_low	65
parity_bit	1
data_in	8912
byte_high	89
byte_low	12
parity_bit	1
data_in	0001
byte_high	00
byte_low	01
parity_bit	0
data_in	060d
byte_high	06
byte_low	0d
parity_bit	0
data_in	3b76
byte_high	3b
byte_low	76
parity_bit	0
data_in	1e3d
byte_high	1e
byte_low	3d
parity_bit	0
data_in	76ed
byte_high	76
byte_low	ed
parity_bit	0
data_in	468c
byte_high	46
byte_low	8c
parity_bit	1
data_in	7cf9
byte_high	7c
byte_low	f9
parity_bit	1
data_in	e3c6
byte_high	e3
byte_low	c6
parity_bit	0
data_in	e2c5
byte_high	e2
byte_low	c5
parity_bit	1
data_in	d5aa
byte_high	d5
byte_low	aa
parity_bit	0
data_in	72e5
byte_high	72
byte_low	e5
parity_bit	0
data_in	bbd7
byte_high	bb
byte_low	d7
parity_bit	0
data_in	8912
byte_high	89
byte_low	12
parity_bit	1
data_in	478f
byte_high	47
byte_low	8f
parity_bit	0
data_in	79f2
byte_high	79
byte_low	f2
parity_bit	0
data_in	e7ce
byte_high	e7
byte_low	ce
parity_bit	0
data_in	f4e8
byte_high	f4
byte_low	e8
parity_bit	0
data_in	e2c5
byte_high	e2
byte_low	c5
parity_bit	0
data_in	2e5c
byte_high	2e
byte_low	5c
parity_bit	0
data_in	debd
byte_high	de
byte_low	bd
parity_bit	0
data_in	962d
byte_high	96
byte_low	2d
parity_bit	0
data_in	b265
byte_high	b2
byte_low	65
parity_bit	0

Task two

```

module Reg_file(
    input clk, write_en,
    input [2:0] read_add, write_add,
    input [15:0] write_data,
    output reg [15:0] read_data,
    output [7:0] read_data_top_byte
);

    reg [15:0] register_file [7:0];
    // memory array with 8 elements 16 bit each

    assign read_data_top_byte = read_data [15:8];

```

```

always @(*) begin
    read_data = register_file [read_add];
end

always @(posedge clk) begin
    if (write_en) begin
        register_file [write_add] <= write_data;
    end
end

endmodule

```

```

module Reg_file_tb;

    reg clk, write_en;
    reg [2:0] read_add, write_add;
    reg [15:0] write_data;
    wire [15:0] read_data;
    wire [7:0] read_data_top_byte;

    Reg_file dut (
        clk, write_en, read_add, write_add,
        write_data, read_data, read_data_top_byte
    );

    initial begin
        clk = 0;
        forever begin
            #10 clk = ~clk;
        end
    end

    initial begin
        $readmemh("mem_file.mem", dut.register_file);

        $display("-----");
        $display(" Time | WE | R_ADD | W_ADD |   W_DATA   |   R_DATA   | TOP");
        $display("_BYTE");
        $display("-----");
    end
endmodule

```

```

$monitor("%5t | %2b | %5d | %5d | %10h | %10h | %8h",
        $time,
        write_en,
        read_add,
        write_add,
        write_data,
        read_data,
        read_data_top_byte);

// THIS TABLE IS AI GENERATED :)

repeat (2) begin
    write_en = 0;
    read_add = $random;
    write_add = $random;
    write_data = $random;

    @(negedge clk);
end

repeat (25) begin
    write_en = 1;
    read_add = $random;
    write_add = $random;
    write_data = $random;

    @(negedge clk);
end

$finish;

end

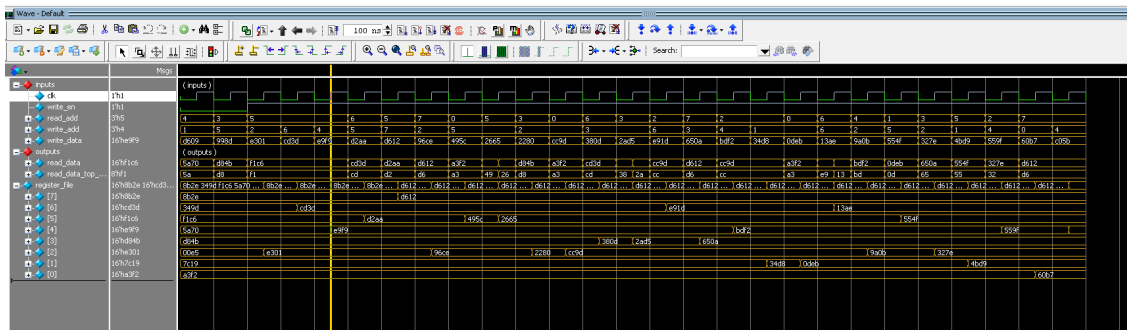
endmodule

```

```

V$IM 20> run -all
#
#-----#
# Time | WE | R_ADD | W_ADD | W_DATA | R_DATA | TOP_BYTE
#-----#
# 0 | 00 | 4 | 1 | 000000d609 | 0000005a70 | 0000005a
# 20 | 00 | 3 | 5 | 000000998d | 000000d84b | 000000d8
# 40 | 01 | 5 | 2 | 000000e301 | 000000f1c6 | 000000f1
# 60 | 01 | 5 | 6 | 000000cd3d | 000000f1c6 | 000000f1
# 80 | 01 | 5 | 4 | 000000e9f9 | 000000f1c6 | 000000f1
# 100 | 01 | 6 | 5 | 000000d2aa | 000000cd3d | 000000cd
# 120 | 01 | 5 | 7 | 000000d612 | 000000d2aa | 000000d2
# 140 | 01 | 7 | 2 | 00000096ce | 000000d612 | 000000d6
# 160 | 01 | 0 | 5 | 000000495c | 000000a3f2 | 000000a3
# 180 | 01 | 5 | 5 | 0000002665 | 000000495c | 00000049
# 190 | 01 | 5 | 5 | 0000002665 | 0000002665 | 00000026
# 200 | 01 | 3 | 2 | 0000002280 | 000000d84b | 000000d8
# 220 | 01 | 0 | 2 | 000000cc9d | 000000a3f2 | 000000a3
# 240 | 01 | 6 | 3 | 000000380d | 000000cd3d | 000000cd
# 260 | 01 | 3 | 3 | 0000002ad5 | 000000380d | 00000038
# 270 | 01 | 3 | 3 | 0000002ad5 | 0000002ad5 | 0000002a
# 280 | 01 | 2 | 6 | 000000e91d | 000000cc9d | 000000cc
# 300 | 01 | 7 | 3 | 000000650a | 000000d612 | 000000d6
# 320 | 01 | 2 | 4 | 000000bdf2 | 000000cc9d | 000000cc
# 340 | 01 | 2 | 1 | 00000034d8 | 000000cc9d | 000000cc
# 360 | 01 | 0 | 1 | 0000000deb | 000000a3f2 | 000000a3
# 380 | 01 | 6 | 6 | 00000013ae | 000000e91d | 000000e9
# 390 | 01 | 6 | 6 | 00000013ae | 00000013ae | 00000013
# 400 | 01 | 4 | 2 | 0000009a0b | 000000bdf2 | 000000bd
# 420 | 01 | 1 | 5 | 000000554f | 0000000deb | 0000000d
# 440 | 01 | 3 | 2 | 000000327e | 000000650a | 00000065
# 460 | 01 | 5 | 1 | 0000004bd9 | 000000554f | 00000055
# 480 | 01 | 2 | 4 | 000000559f | 000000327e | 00000032
# 500 | 01 | 7 | 0 | 00000060b7 | 000000d612 | 000000d6
# 520 | 01 | 7 | 4 | 000000c05b | 000000d612 | 000000d6
# ** Note: $finish : D:/NTI - Digital IC Design/Session 4/Assignment/Register file/Reg_file_tb.v(62)
# Time: 540 ns Iteration: 1 Instance: /Reg_file_tb
# 1
# Break in Module Reg_file_tb at D:/NTI - Digital IC Design/Session 4/Assignment/Register file/Reg_file_tb.v line 62
V$IM 21>

```



Task 3

```

module grid_mem_router #(
    parameter WORD_WIDTH = 32
)(
    input clk, rst, out_en, endian_swap, col_add,
    input [1:0] row_add,
    output reg [WORD_WIDTH - 1:0] processing_word,
    inout [7:0] bus_data
);

    reg [7:0] fabric_mem [3:0] [1:0];

```

```

// grid memory with 4 rows and 2 columns each address is 8 bits wide

assign bus_data = out_en? fabric_mem[row_add][col_add] : 8'hzz;
// tri state buffer logic

integer i;

always @(posedge clk or posedge rst) begin

    if (rst) begin
        processing_word <= 0;
    end else begin

        for (i = 0; i < 4; i = i+1) begin
            if (endian_swap) begin
                processing_word[(WORD_WIDTH - 8*i - 1) -: 8] <= fabric_mem
[i] [0];
                // endianness swap
            end else begin
                processing_word[8*i +: 8] <= fabric_mem [i] [0];
                // copying the first column
            end
        end

    end

end

end

endmodule

```

```

module grid_mem_router_tb;
    reg clk, rst, out_en, endian_swap, col_add;
    reg [1:0] row_add;
    wire [dut.WORD_WIDTH - 1:0] processing_word;
    wire [7:0] bus_data;

    grid_mem_router dut (
        .clk(clk),
        .rst(rst),
        .out_en(out_en),
        .endian_swap(endian_swap),
        .col_add(col_add),

```

```

        .row_add(row_add),
        .processing_word(processing_word),
        .bus_data(bus_data)
    );

initial begin
    dut.fabric_mem[0][0] = 8'hAA;
    dut.fabric_mem[0][1] = 8'h11;

    dut.fabric_mem[1][0] = 8'hBB;
    dut.fabric_mem[1][1] = 8'h22;

    dut.fabric_mem[2][0] = 8'hCC;
    dut.fabric_mem[2][1] = 8'h33;

    dut.fabric_mem[3][0] = 8'hDD;
    dut.fabric_mem[3][1] = 8'h44;

    // filling the memory with dummy data 3shan maytla3sh unkown values X
end

initial begin
    clk = 0;
    forever begin
        #10 clk = ~clk;
    end
end

initial begin
    rst = 1;
    out_en = 0;
    endian_swap = 0;
    col_add = 0;
    row_add = 0;

    @(negedge clk);

    rst = 0;

    repeat(25) begin
        out_en = $random;
        endian_swap = $random;
        col_add = $random;
        row_add = $random;
    end
end

```

```

        @(negedge clk);
    end

    $finish;

end
endmodule

```

